Detect, Then Refine: A YOLO11s and U-Net Pipeline for Solar Filament Segmentation in the MAGFiLO 2026 Challenge

Boobathi V, Chandru S, Ahamed Jaseem J and Kajamohinudeen A K
Department of Computer Science and Engineering, SSN College of Engineering, Chennai, India

***Abstract*—Solar filaments are dark, thread-like structures seen in $\mathrm{H}\alpha$ images of the Sun. When they erupt, they can trigger space-weather events that disturb satellites and power grids, so keeping an automatic eye on them is genuinely useful. The MAGFiLO 2026 challenge asks for one pixel-exact mask for every filament in a full-disk image, submitted as run-length-encoded (RLE) strings. Our solution is a simple two-step idea: first a YOLO11s detector looks at the whole Sun and says where the filaments are, then a U-Net zooms into each detected region and carefully marks the exact pixels that belong to that filament. The U-Net is given a rectangular "seed" marking the detected box so it always knows which filament it is outlining. On 106 held-out images (843 filaments) the detector reaches mAP50 $= 0.618$, and the refiner reaches a Dice of 0.822 when given correct boxes. On the 180 test images the system outputs 1,212 masks, about 6.73 per image. Our analysis shows that detector recall, not mask quality, is the main bottleneck.**

***Index Terms*—solar filaments, H-alpha, instance segmentation, YOLO11, U-Net, space weather, MAGFiLO**

I. Introduction

The Sun is far from quiet. Its magnetic field constantly lifts cool, dense gas into the hot corona, and when a cloud of such gas is seen against the bright solar disk in $\mathrm{H}\alpha$ light, it appears as a dark thread called a filament. When a filament becomes unstable it can erupt, and these eruptions are closely linked to coronal mass ejections that can affect satellites, radio communications and power grids. Watching filaments automatically, every day and across the whole disk, is therefore a small but meaningful piece of space-weather monitoring. The MAGFiLO dataset [1] provides expert-drawn outlines of filaments, and the Kaggle MAGFiLO 2026 challenge [2] turns this into a competition.

The task, in plain words: given a single full-disk $\mathrm{H}\alpha$ image of size 2048×2048, find every filament and outline each one separately, pixel by pixel, as a set of non-overlapping binary masks.

Why this is harder than it sounds. Filaments are thin and low-contrast, so they easily vanish when a huge 2048^2 image is shrunk to fit a neural network. The disk is also full of other dark things (sunspots, noise, fine fibrils) that are not annotated, so "dark pixel" is a poor rule. The number of filaments changes from image to image, and the evaluation punishes both missed and extra masks. Finally, we noticed that the annotation file lists 1,154 image entries but only 707 actual image files, which can quietly leak validation data into training if the split is done carelessly.

Our approach. A small YOLO11s detector answers *where are the filaments?*, and a seed-conditioned U-Net answers *which exact pixels belong to each one?*, working on a zoomed-in crop around every detected box.

This paper makes the following contributions.

1) A fully reproducible *detect-then-refine* pipeline that turns a full-disk $\mathrm{H}\alpha$ image into disjoint, RLE-encoded filament instance masks (Fig. 1).
2) A *seed-conditioned* U-Net refiner: a rectangle marking the detected box is supplied as an extra input channel, so the network outlines only the requested filament even when several share a crop.
3) A *leak-free* file-level split that removes the duplicate-entry problem of the annotation file, with box-jitter training so the refiner tolerates imperfect detector boxes.
4) An honest error analysis showing that detector recall, not mask quality, limits the pipeline.

Section II reviews related work and the research gap, Section III describes the data and the system, Section IV the results, and Sections V and VI the limitations and conclusions.

II. Literature Survey

A. Classical filament extraction

Shih and Kowalski [3] addressed automatic filament extraction from $\mathrm{H}\alpha$ solar images. Their solution uses intensity thresholding to find dark regions, followed by region growing and morphological operations to clean up the result. It is fast and interpretable, but depends on hand-tuned thresholds and cannot separate filaments from other dark features. Qu et al. [4] proposed an automatic detection method that combines region growing with edge information to segment filaments and group them into coherent structures. Like other classical methods, it relies on fixed thresholds and instrument-specific tuning, so it does not generalise well. Bernasconi et al. [5] built an automated detection and characterisation code for full-disk $\mathrm{H}\alpha$ images. It chains intensity-based segmentation with morphological filtering, and then measures properties such as length, area and chirality of each filament. It improves on plain thresholding, but brightness-based rules still cannot tell a filament from a sunspot or noise that looks equally dark.

B. Learning-based recognition

Zhu et al. [6] were among the first to apply deep learning to filament recognition in $\mathrm{H}\alpha$ images. Their solution trains

a convolutional neural network to classify image patches as filament or non-filament, replacing hand-made rules with learned features. Its accuracy was limited by the small amount of labelled data available then, which is exactly the gap the MAGFiLO dataset fills today.

C. Building blocks we use

YOLO11 [7] is a fast single-stage detector. U-Net [8] is an encoder–decoder segmentation network with skip connections, here used with a ResNet encoder [9]. The Dice loss [10] directly optimises overlap and suits thin foreground structures. Prompting a segmentation model with a box, as in the Segment Anything Model [11], motivates our seed channel, although our refiner is small and trained specifically on Hα filaments.

D. Research gap and questions

The classical methods [3]–[5] are fast but confuse filaments with look-alike dark features; the learning-based method [6] shows the value of learned features but was held back by limited labels and works at patch level, not at the level of individual instance masks. In the sources we reviewed, we did not find a full-disk pipeline that produces one disjoint mask per filament and is trained on the large MAGFiLO label set. Our work combines both lessons: a learned detector for robust localisation and a learned refiner for exact masks. We ask two questions. **RQ1:** can a small detector plus a seed-conditioned refiner produce a valid instance-mask submission on the full-disk images? **RQ2:** which stage limits the pipeline, localisation or mask quality?

III. Methodology

Our complete pipeline is written as a single, self-contained Kaggle notebook (later refactored into ten modular scripts) that runs from raw annotations to `submission.csv`. This section follows the order in which the code executes: finding the data, analysing it, preparing it, training the two models, and finally generating the submission. Fig. 1 shows the whole system.

A. Problem formulation

Let $I \in \{0,\dots,255\}^{2048\times2048}$ be a grayscale full-disk image. The goal is a set of $K(I)$ binary masks $\{M_k\}_{k=1}^{K}$ with $M_k \in \{0,1\}^{2048\times2048}$ that are pairwise disjoint,

$$M_j \cap M_k = \varnothing \quad \text{for } j \neq k, \tag{1}$$

each encoded as an RLE string. The pipeline factorises into a detector $D(I) = \{(b_k, s_k)\}$ returning boxes $b_k = [x, y, w, h]$ with confidence s_k, and a refiner R that maps a crop of I around b_k and a seed of b_k to a probability map. Overlap quality of a predicted mask $\hat{M}$ against the expert mask M is measured with the Dice coefficient,

$$\mathrm{Dice}(\hat{M}, M) = \frac{2\,|\hat{M} \cap M|}{|\hat{M}| + |M|}. \tag{2}$$

TABLE I
Leak-free file-level split (seed 42, 15% validation).

Split	Files	COCO entries	Annotations
Train	601	984	6,941
Validation	106	170	1,258
Total	707	1,154	8,199

B. Step 1: Finding the data

Rather than hard-coding file paths, the notebook searches the whole `/kaggle/input` directory automatically. It looks for (i) the COCO-format training annotation JSON, (ii) the training image folder, and (iii) the test image folder, by name matching ("train", "test", "annotation", "magfilo"). It then scans the directories to count images. This found exactly one annotation file, 707 training images and 180 test images, so the pipeline works even if the dataset is mounted under a slightly different path.

C. Step 2: Exploring the data

Before training anything, we loaded the COCO JSON and counted what was inside. Three findings shaped the rest of the pipeline.

- **Size.** The training annotations contain 1,154 image entries and 8,199 filament annotations, i.e. about 7.10 filaments per image on average.
- **Duplicates.** There are only 707 distinct image files for those 1,154 entries, meaning the same physical image appears several times in the annotation file. Splitting by annotation entry (rather than by file) would put near-copies of the same Sun photo in both training and validation and inflate our scores.
- **Nothing missing.** Every one of the 1,154 entries could be matched to a real image file on disk (with a fallback that matches by file name and then by file stem), so no training data was silently dropped.

Annotations are stored either as polygons or as RLE; crowd annotations and entries with neither a segmentation nor a box are skipped, and when a segmentation cannot be decoded the box is used as a rectangular fallback mask.

D. Step 3: Pre-processing and the leak-free split

Splitting by file. We collect the unique file stems, shuffle them with seed 42, and hold out 15% for validation. This gives 601 training files (984 COCO entries, 6,941 annotations) and 106 validation files (170 entries, 1,258 annotations), as summarised in Table I. No file appears on both sides.

Detector labels. For YOLO, each filament's bounding box $[x, y, w, h]$ is clamped to the image borders (with a minimum size of 1 px so degenerate boxes cannot crash training) and converted to the normalised YOLO format $(0,\ x_c/W,\ y_c/H,\ w/W,\ h/H)$ centred on the box. Each image gets one text file with one line per filament, all under a single class 0: filament, because the competition only needs masks, not filament types. One label file is written per physical

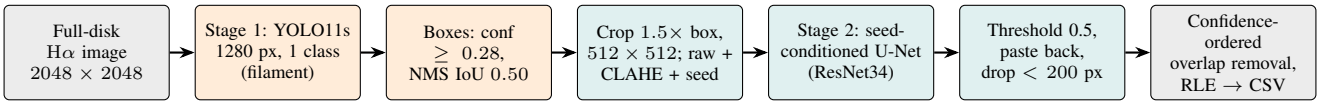


Fig. 1. End-to-end system. A YOLO11s detector proposes filament boxes on the whole disk; each box is turned into a zoomed crop with a seed channel, a U-Net refiner outputs the exact pixels, and masks are made disjoint before RLE encoding.

image, using only the first annotation entry per file; this is why the 106 validation files contain 843 detector boxes rather than 1,258.

Refiner samples. For the U-Net, each annotation becomes a training sample consisting of a square crop around the filament, the same crop after contrast enhancement, and a binary "seed" image (Section III-E). The target is the expert polygon mask, decoded from COCO format.

Augmentation. The detector uses Ultralytics' built-in augmentation (mosaic, horizontal flips, scaling, translation, HSV jitter), with mosaic automatically switched off for the last 10 epochs. For the refiner we use horizontal and vertical flips (applied jointly to the image, the enhanced image, the seed and the target), plus box jitter: during training the input box is randomly shifted by up to $\pm 15\%$ of its size and scaled by a factor in $[0.85, 1.25]$ per axis, and the seed padding is drawn uniformly from $[0.2, 0.6]$ instead of the fixed 0.4. The U-Net therefore learns to be tolerant of imperfect detector boxes at test time.

E. Step 4: Stage 1 — how YOLO finds the filaments

The first model is YOLO11s [7], a small and fast object detector pre-trained on COCO. In simple terms, YOLO looks at the image once and divides it into a grid; each grid cell proposes a few candidate boxes together with a confidence score saying how sure it is that a filament sits inside. Because many cells can propose boxes around the same filament, a second step, non-maximum suppression (NMS), keeps only the best-scoring box when several boxes overlap heavily, removing the duplicates. What remains is a list of boxes, each with a confidence value.

We train at 1280 px rather than the default 640 px. This matters: the input images are 2048^2, and thin filaments almost disappear if the image is squeezed too aggressively. At inference we keep boxes with confidence ≥ 0.28 and apply NMS with an IoU threshold of 0.50.

We deliberately chose the small YOLO variant (9.4M parameters). It trains in under 20 minutes on two Tesla T4 GPUs, and precision is not its job anyway; that is left to the next stage.

F. Step 5: Stage 2 — how the U-Net finds the exact pixels

A bounding box is not a mask. Filaments are long, thin and often diagonal, so most of a box is actually background (Fig. 2). The second stage turns each box into a careful pixel-level outline.

For every box $[x, y, w, h]$ we cut out a square context crop centred on the box with side

$$s = \min\big(\max(\lceil 1.5 \max(w, h) \rceil,\ 96),\ 2048\big), \tag{3}$$

shift it back inside the image if it crosses a border, and resize it to 512×512. Because the crop is small, the filament occupies many more pixels than it did in the full-disk view, which is effectively a several-fold zoom. The U-Net [8] with a ResNet34 encoder [9] then receives a three-channel input:

1) the raw grayscale crop,
2) a CLAHE-enhanced version of the crop (clip limit 2.0, 8×8 tiles), which brightens faint filaments,
3) the seed channel: a white rectangle covering the detected box, enlarged by 40% of its width and height on every side.

All channels are scaled to $[0, 1]$ and normalised with ImageNet statistics. The seed is the key idea. A plain U-Net would happily outline every dark thread in the crop. The seed acts like a pointer: it tells the network "the filament you care about lives inside this rectangle", so when two filaments share a crop, the network still outlines only the requested one. We therefore call it a *seed-conditioned* refiner. This is the same spirit as prompting SAM with a box [11], but our refiner is small and trained specifically on Hα filaments.

The U-Net itself is an encoder–decoder network: the encoder (ResNet34, pre-trained on ImageNet) gradually compresses the crop into features, and the decoder expands them back to full resolution, with skip connections passing fine detail from encoder to decoder so thin boundaries survive. Its output is not a yes/no label but a probability map: for every one of the 512×512 pixels, a number $p \in [0, 1]$ saying how confident the network is that the pixel belongs to the filament. We simply threshold this map at 0.5. Training minimises a blend of binary cross-entropy and soft Dice loss [10],

$$\mathcal{L} = 0.45\, \mathcal{L}_{\mathrm{BCE}} + 0.55\, \mathcal{L}_{\mathrm{Dice}}, \tag{4}$$

which balances pixel accuracy with overall overlap. The checkpoint with the highest validation Dice is kept.

G. Step 6: Post-processing and building the submission

Each crop mask is resized back with nearest-neighbour interpolation and pasted onto a full 2048×2048 canvas at its original location, and connected components smaller than 200 px are discarded as noise. Since neighbouring filaments can produce overlapping masks, we process boxes in descending confidence order: a more confident mask keeps any pixel it shares with a less confident one, and masks that become empty are dropped. Formally,

$$\hat{M}_k = M_k \setminus \bigcup_{j<k} \hat{M}_j, \tag{5}$$

which guarantees the final masks are disjoint. Finally every mask is run-length encoded with `pycocotools` and written

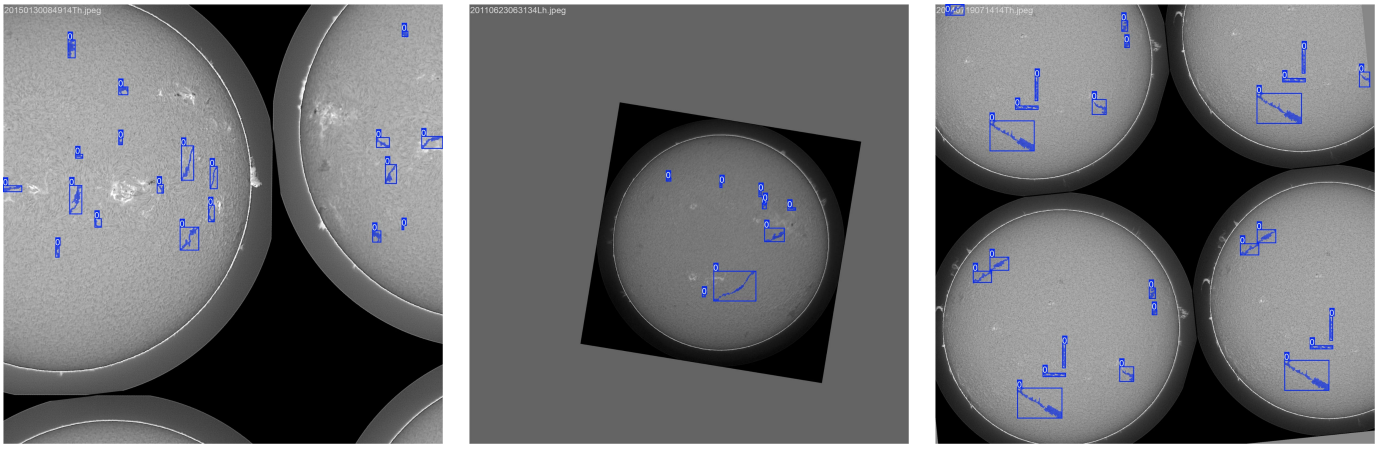


Fig. 2. Detector output of our YOLO11s pipeline (class 0 = filament). (a) Multi-disk mosaic tile with many detections. (b) Rotated, rescaled disk on padded background. (c) Four-tile mosaic of one disk with flips. The strong augmentation produces varied views, and the long, thin, diagonal filaments inside large boxes show why a box alone cannot serve as a mask, which is exactly why the refiner stage exists.

TABLE II
MAIN HYPER-PARAMETERS OF THE TWO STAGES.

Setting	Value
Detector	YOLO11s, COCO pre-trained
Detector input size	1280 px
Detector epochs / batch	40 / 8
Inference confidence / NMS IoU	0.28 / 0.50
Refiner	U-Net, ResNet34, ImageNet init
Refiner crop	512×512 (context $1.5\times$, min 96 px)
Seed padding	40% per side (jitter 20–60%)
Refiner epochs / batch	12 / 8
Refiner optimiser	AdamW, lr 3×10^{-4}, wd 10^{-4}
Refiner loss	0.45 BCE $+ 0.55$ soft Dice
Mask threshold / min component	0.5 / 200 px
Seed	42

as a row `<image_id>_<k>` into `submission.csv`. Test images are de-duplicated by file name and resized to 2048×2048 if needed.

IV. EXPERIMENTS AND RESULTS

A. Training setup

Everything runs in one Kaggle notebook with seeds fixed to 42. The detector trains for 40 epochs (batch 8, AdamW chosen automatically by Ultralytics, patience 10 not triggered) on two Tesla T4 GPUs using distributed data parallel. The refiner trains for 12 epochs (batch 8, AdamW with learning rate 3×10^{-4} and weight decay 10^{-4}, cosine annealing, mixed precision). Table II lists the main settings.

B. Detector results

Table III reports the final detector on the held-out files. Training longer clearly helped: mAP50 rose from 0.548 at epoch 15 to 0.617 at epoch 40, and the last-ten-epoch (no-mosaic) phase climbed steadily from 0.560 to 0.617, peaking at 0.625 at epoch 39. The gap between mAP50 (0.618) and mAP50–95 (0.370) tells us the boxes are usually in the right place but a bit loose at strict overlap, which is expected for long thin objects and exactly what the refiner's seed and box jitter absorb.

TABLE III
DETECTOR RESULTS ON HELD-OUT FILES (106 IMAGES, 843 FILAMENTS).

Checkpoint	P	R	mAP50	mAP50–95	F1
Epoch 15	0.543	0.585	0.548	0.302	0.563
Epoch 40 (last)	0.591	0.617	0.617	0.371	0.604
`best.pt` (final)	0.593	0.617	0.618	0.370	0.605

C. Refiner results

The refiner's validation Dice improved every epoch, from 0.786 at epoch 1 to 0.822 at epoch 12, while training and validation Dice ended within 0.005 of each other, with no sign of over-fitting. Importantly, this Dice is measured by feeding the U-Net the ground-truth boxes as seeds, so it is an optimistic, upper-bound-style measure of mask quality, not an end-to-end score.

D. Test output

On the 180 test images the full pipeline produced 1,212 masks in a 0.35 MB `submission.csv` (predictions on 178 images; two images received none). That is 6.73 masks per image against 7.10 annotated per training image, which is the right regime and slightly conservative, though a matching count alone does not prove accuracy. This answers RQ1: the pipeline yields a valid, disjoint, RLE-encoded submission.

E. Error budget: what actually limits us

The pipeline is serial, so errors compound: a filament the detector misses is gone forever. Multiplying detector recall

(0.617) by refiner Dice (0.822) gives a rough overlap estimate of

$$0.617 \times 0.822 \approx 0.51, \tag{6}$$

which is not a measured metric, but is enough to make the point clearly: detector recall, not mask quality, is the bottleneck (RQ2). Raising recall (a larger detector, higher resolution, or tiled inference) should help more than further polishing the U-Net.

V. Limitations and Future Work

We have no end-to-end, mask-level validation score: the detector is scored by mAP50 on boxes and the refiner by Dice with ground-truth seeds, and the product above is only a rough guide. Detector labels use only the first annotation entry per image file, so the 843 validation boxes undercount the 1,258 annotated filaments. The confidence threshold, mask threshold and minimum component size were fixed by hand rather than tuned on validation data. Two test images received no predictions, and we report no repeated runs or confidence intervals.

Future work: an end-to-end mask-level validation score, tuning the confidence and threshold values on validation data, and higher-recall detection (a larger detector, higher input resolution or tiled inference).

VI. Conclusion

We built a simple, fully reproducible detect-then-refine pipeline for solar filament instance segmentation. A small YOLO11s finds the filaments (mAP50 0.618 on held-out files), a seed-conditioned ResNet34 U-Net outlines each one at high effective resolution (Dice 0.822 with ground-truth seeds), and confidence-ordered overlap resolution plus RLE encoding produces a valid submission of 1,212 masks. Honest analysis shows detector recall is the main bottleneck.

Acknowledgment

We thank the authors of the MAGFiLO dataset, the organisers of the MAGFiLO 2026 challenge, and Kaggle for the GPU resources used in this work.

References

[1] A. Ahmadzadeh et al., "A dataset of manually annotated filaments from H-alpha observations," *Scientific Data*, vol. 11, 2024.

[2] MAGFiLO 2026, "Solar Filament Segmentation Challenge," Kaggle competition, 2026. [Online]. Available: https://www.kaggle.com/competitions/filament-segmentation-2026

[3] F. Y. Shih and A. J. Kowalski, "Automatic extraction of filaments in Hα solar images," *Solar Physics*, vol. 218, pp. 99–122, 2003.

[4] M. Qu, F. Y. Shih, J. Jing, and H. Wang, "Automatic solar filament detection using image processing techniques," *Solar Physics*, vol. 228, pp. 119–135, 2005.

[5] P. N. Bernasconi, D. M. Rust, and D. Hakim, "Advanced automated solar filament detection and characterization code," *Solar Physics*, vol. 228, pp. 97–117, 2005.

[6] G. Zhu et al., "Solar filament recognition based on deep learning," *Solar Physics*, vol. 294, art. 117, 2019.

[7] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[10] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. 3DV*, 2016, pp. 565–571.

[11] A. Kirillov et al., "Segment anything," in *Proc. IEEE ICCV*, 2023, pp. 4015–4026.